DESIGN AND APPLY CONTEXT-AWARE ACCESS CONTROLS BASED ON USER IDENTITY, DEVICE POSTURE, LOCATION, AND APPLICATION RISK.

INTRODUCTION:

In today's highly connected digital environment, organizations depend heavily on online systems, cloud platforms, and remote access to perform daily operations. As businesses adopt modern technologies such as cloud computing, mobile devices, and remote working environments, ensuring secure access to organizational resources has become increasingly complex. Traditional security models that rely only on network boundaries and static authentication methods are no longer sufficient to protect sensitive data and critical applications from cyber threats.

The concept of context-aware access control has emerged as an effective solution to address these challenges. Unlike traditional access control mechanisms that simply verify a user's identity through a username and password, context-aware systems analyse multiple factors before granting access to a resource. These factors include the user's identity and role within the organization, the security status of the device being used, the geographical location of the access request, and the risk level associated with the application or data being accessed.

By evaluating these contextual attributes in real time, the system can make intelligent decisions about whether access should be allowed, restricted, or denied. For instance, if a trusted employee accesses a standard application using a secure company laptop from a recognized location, the system may allow seamless access. However, if the same user attempts to access a high-risk application from an unknown device or unfamiliar location, the system may require additional authentication steps or block the request entirely.

The primary goal of this project is to design and implement a context-aware access control framework that enhances security while maintaining usability. By integrating multiple contextual signals into the access decision process, the system helps organizations reduce security risks, prevent unauthorized access, and protect sensitive information. At the same time, it ensures that legitimate users can access necessary resources efficiently, supporting a secure and flexible digital working environment.

ABSTRACT:

The rapid growth of cloud technologies, mobile devices, and remote working environments has significantly changed the way organizations access and manage digital resources. In this modern environment, the traditional network boundary has become less relevant, making conventional access control methods insufficient for protecting sensitive information. As users access systems from different devices, locations, and networks, organizations face increased security risks such as unauthorized access, data breaches, and compromised user credentials.

To address these challenges, this project focuses on designing and implementing a context-aware access control system that evaluates multiple factors before granting access to applications and data. Instead of relying only on basic authentication methods, the system considers several contextual parameters including user identity, device security posture, geographic location, and the risk level of the requested application. By analysing these parameters together, the system can determine whether the access request is safe or potentially risky.

The proposed system uses a policy-based decision mechanism that dynamically evaluates each access request in real time. Based on the calculated risk level, the system can allow full access, request additional authentication such as multi-factor authentication (MFA), restrict certain actions, or completely deny access. This adaptive approach helps ensure that trusted users experience smooth access while potentially harmful requests are carefully controlled or blocked.

The main goal of this project is to create a flexible and intelligent security framework that strengthens organizational protection without affecting user productivity. By implementing context-aware decision-making, the system reduces the chances of unauthorized access and improves overall security management in modern digital infrastructures.

Use Case Scenarios:

Scenario 1: Finance Employee Accessing ERP Application

A senior accountant needs to access the company's ERP system for payroll processing from home using a company-managed laptop. After logging in through Azure Active Directory with MFA, the system verifies the user's identity, group membership, and device security. Once verified, a secure micro-tunnel is created to the ERP application. The user can access only the ERP system and cannot view other internal resources like development servers or HR databases.

Output:

- User identity and role verified successfully
- Secure micro-tunnel created only for the ERP application
- Other internal applications remain hidden and inaccessible

Scenario 2: Temporary Contractor Access to Project Repository

A software contractor working on a short-term project needs access to a specific Git repository. The contractor is added as a guest user and assigned to the Project_X_Contractors group. When connecting through the ZPA client, the system verifies the user's identity and device security status. According to the policy, access is granted only to the Git repository during working hours via SSH or HTTPS. The contractor can manage code but cannot access other internal resources like file servers or company portals.

Output:

- Access limited to project-related repository
- Time-based restrictions applied
- Movement to other internal systems blocked

Scenario 3: Executive Access to Confidential Documents

A company executive needs to access confidential SharePoint documents while traveling abroad using a company tablet. After MFA authentication through Azure Active Directory, the system

checks the user's identity, privileges, device compliance, and location. Due to the unusual location, access is allowed with restrictions such as view-only mode, and the security team is alerted about the high-risk login.

Output:

- Authentication completed with MFA
- Access granted with restricted permissions (view-only)
- Security team notified about unusual login location

Scenario 4: IT Technician Accessing Support Tools

An IT support technician connects from home using a company-managed desktop and authenticates through Azure Active Directory with MFA. The system verifies the technician's role, device security, and location. Since the context is trusted, access is granted to support tools like ServiceNow and the remote desktop host, while other systems such as finance and HR remain restricted.

Output:

- Technician identity and role verified
- Secure micro-tunnel created for ServiceNow and remote desktop host
- Access limited to support-related applications only

Scenario 5: Marketing Staff Accessing CRM from Public Network

A marketing employee accesses the Salesforce CRM system from a coffee shop using a company-managed smartphone and authenticates with MFA through Azure Active Directory. Although the user and device are verified, the public Wi-Fi is considered risky. Therefore, the system allows access through secure browser isolation to protect sensitive CRM data.

Output:

- User authenticated and device verified
- High-risk network detected
- Secure browser isolation used to protect sensitive data

Stage 1

Title of the project:

Design and apply context-aware access controls based on user identity, device posture, location, and application risk.

Overview:

In today's digital environment, organizations rely on cloud services, mobile devices, and remote working systems to perform daily operations. Employees and users often access company resources from different locations and devices, which increases flexibility but also creates security risks. Traditional security models mainly depend on network-based access control and simple authentication methods, which are no longer sufficient to protect sensitive data.

To address this issue, organizations are adopting context-aware access control systems. These systems evaluate multiple factors such as user identity, device posture, location, and application risk before granting access to resources. This project focuses on designing and applying such a system to improve security and ensure that only authorized users can access specific applications. The project follows the Zero Trust security principle, which means every access request must be verified before permission is granted. The system checks the user's identity, the security condition of the device, and the location of the access request. It also considers the sensitivity of the application being accessed.

Based on these contextual factors, the system can allow access, require additional authentication, restrict certain actions, or deny the request if the risk level is high. This approach ensures that users receive only the minimum level of access required, reducing the chances of unauthorized access and data breaches.

Overall, this project aims to develop a secure and efficient access control framework that enhances organizational security while supporting modern work environments where users access resources from various devices and locations.

List of teammates:

S.NO	NAME	COLLAGE	CONTACT
1.	ANWAR R	Shree Venkateshwara College of Arts and Science	anwarr2023aids@gmail.com
2.	ARUNKUMAR P	Shree Venkateshwara College of Arts and Science	arunkumarp2023aids@gmail.com
3.	ABISHEK .B	Shree Venkateshwara College of Arts and Science	abishekb2023aids@gmail.com
4.	DEEPAKKUMAR K	Shree Venkateshwara College of Arts and Science	deepakkumark2023aids@gmail.com
5.	DHIYANESHWAR S	Shree Venkateshwara College of Arts and Science	dhiyaneshwars2023aids@gmail.com

List of Vulnerability Table:

S.NO	CATEGORY NAME	VULNERABILITY NAME	CWE/CVE – No
1.	Broken Access Control	Inclusion of Sensitive Information in Source Code	CWE-540
2.	Security Misconfiguration	Improper Restriction of XML External Entity Reference	CWE-611
3.	Software Supply Chain Failures	Reliance on Insufficiently Trustworthy Component	CWE-1357
4.	Cryptographic Failures	Insufficient Entropy	CWE-331

5.	Injection	Public Static Field Not Marked Final	CWE-500
6.	Insecure Design	Violation of Secure Design Principles	CWE-657
7.	Authentication Failures	Use of Hard-coded Credentials	CWE-798
8.	Software or Data Integrity Failures	Replicating Malicious Code (Virus or Worm)	CWE-509
9.	Security Logging and Alerting Failures	Information Loss of Omission	CWE-221
10.	Mishandling of Exceptional Conditions	Omitted Break Statement in Switch	CWE-484

REPORT:

1. Vulnerability Name: Inclusion of Sensitive Information in Source Code

CWE/CVE: 540

OWASP/SANS Category: OWASP Top 10 – Broken Access Control

Description:

This vulnerability occurs when sensitive information is exposed within the application's source code. Developers may accidentally include confidential data such as API keys, database credentials, authentication tokens, or internal system details directly in the source code. If the application’s source code becomes accessible to users or attackers, this information can be easily discovered and misused.

When sensitive data is hardcoded into the application or visible in publicly accessible files, attackers may retrieve the information by simply viewing the page source, accessing configuration files, or examining exposed scripts. This exposure increases the risk of unauthorized access to internal systems, databases, or third-party services.

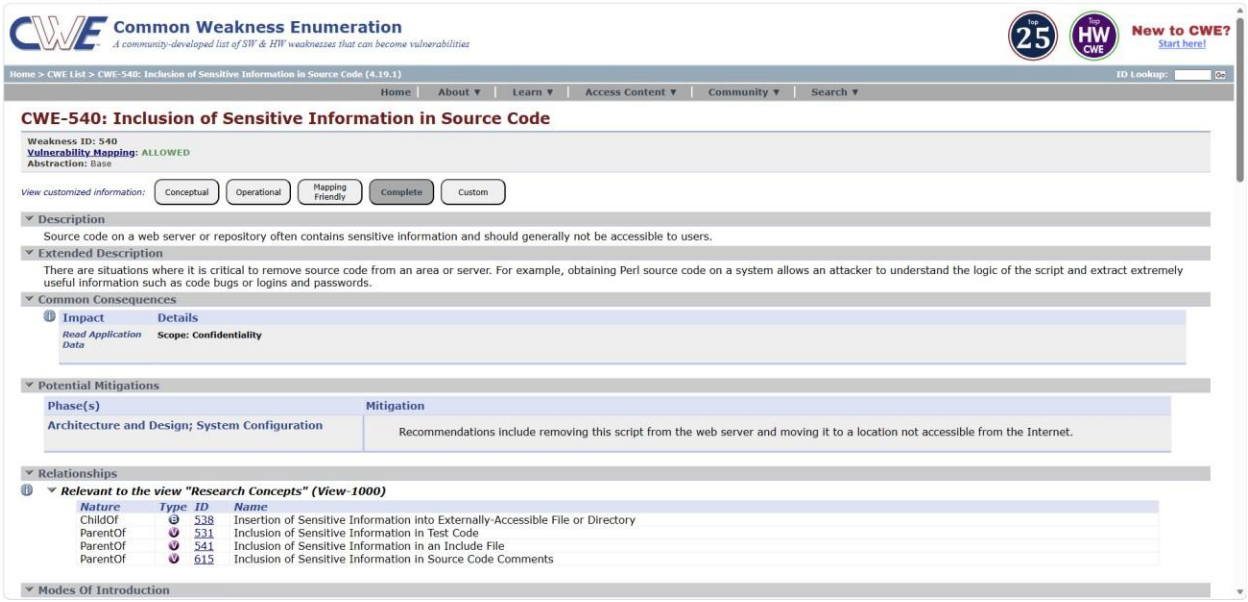
Business Impact:

If sensitive information is included in the source code, attackers may gain access to confidential system details such as database credentials, API keys, or internal endpoints. This could allow attackers to access restricted systems, manipulate application data, or compromise the entire infrastructure. From a business perspective, such exposure may lead to data breaches, unauthorized system access, financial loss, and damage to the organization's reputation. Attackers may also use the exposed credentials to perform malicious actions such as modifying data, extracting sensitive customer information, or launching further attacks against internal services.

Vulnerability Path:
http://testfire.net/search.jsp

Steps to Reproduce:

- Access the web application.
- Right-click on the webpage and select View Page Source or inspect the developer tools.
- Look for sensitive information such as API keys, passwords, tokens, or internal system details inside the HTML, JavaScript, or configuration files.
- If sensitive data is present in the source code, it can be copied and potentially misused by an attacker.



Common Weakness Enumeration
A community-developed list of SW & HW weaknesses that can become vulnerabilities

Home > CWE List > CWE-540: Inclusion of Sensitive Information in Source Code (4.19.1)

Home | About ▾ | Learn ▾ | Access Content ▾ | Community ▾ | Search ▾

CWE-540: Inclusion of Sensitive Information in Source Code

Weakness ID: 540
Vulnerability Mapping: ALLOWED
Abstraction: Base

View customized information: Conceptual | Operational | Mapping Friendly | Complete | Custom

Description

Source code on a web server or repository often contains sensitive information and should generally not be accessible to users.

Extended Description

There are situations where it is critical to remove source code from an area or server. For example, obtaining Perl source code on a system allows an attacker to understand the logic of the script and extract extremely useful information such as code bugs or logins and passwords.

Common Consequences

Impact	Details
Read Application Data	Scope: Confidentiality

Potential Mitigations

Phase(s)	Mitigation
Architecture and Design; System Configuration	Recommendations include removing this script from the web server and moving it to a location not accessible from the Internet.

Relationships

Relevant to the view "Research Concepts" (View-1000)

Nature	Type	ID	Name
ChildOf	V	538	Insertion of Sensitive Information into Externally-Accessible File or Directory
ParentOf	V	531	Inclusion of Sensitive Information in Test Code
ParentOf	V	541	Inclusion of Sensitive Information in an Include File
ParentOf	V	615	Inclusion of Sensitive Information in Source Code Comments

Modes Of Introduction

Recommendation:

- Avoid storing sensitive information directly in source code.
- Store secrets such as API keys and passwords in secure environment variables or secret management systems.
- Implement proper access control mechanisms to restrict access to configuration files and internal resources.
- Regularly review and audit source code to ensure that confidential information is not exposed.
- Use secure vault services or encrypted storage to protect sensitive credentials.

2. Vulnerability name: Improper Restriction of XML External Entity Reference

CWE: 611

OWASP category: OWASP Top 10 – Security Misconfiguration

Description:

This vulnerability occurs when an XML parser is configured to process external entities without proper security restrictions. XML External Entity (XXE) attacks exploit this weakness by allowing attackers to include malicious external references in XML input.

When the application processes the malicious XML file, the external entity may access sensitive files on the server, interact with internal systems, or send confidential data to the attacker. This happens because the XML parser resolves the external entity and includes the referenced data during processing.

Improper configuration of XML parsers, such as allowing external entity processing or failing to disable DTD (Document Type Definition) processing, can make applications vulnerable to XXE attacks.

Business Impact:

If exploited, attackers may gain unauthorized access to sensitive files stored on the server, such as system configuration files, credentials, or internal documents. They may also use this vulnerability to perform Server-Side Request Forgery (SSRF) attacks by forcing the server to make requests to internal or external systems.

From a business perspective, this vulnerability can lead to data leakage, unauthorized system access, and potential compromise of internal infrastructure. In severe cases, attackers may retrieve confidential information such as user credentials or system configuration data, which can result in financial loss and reputational damage.

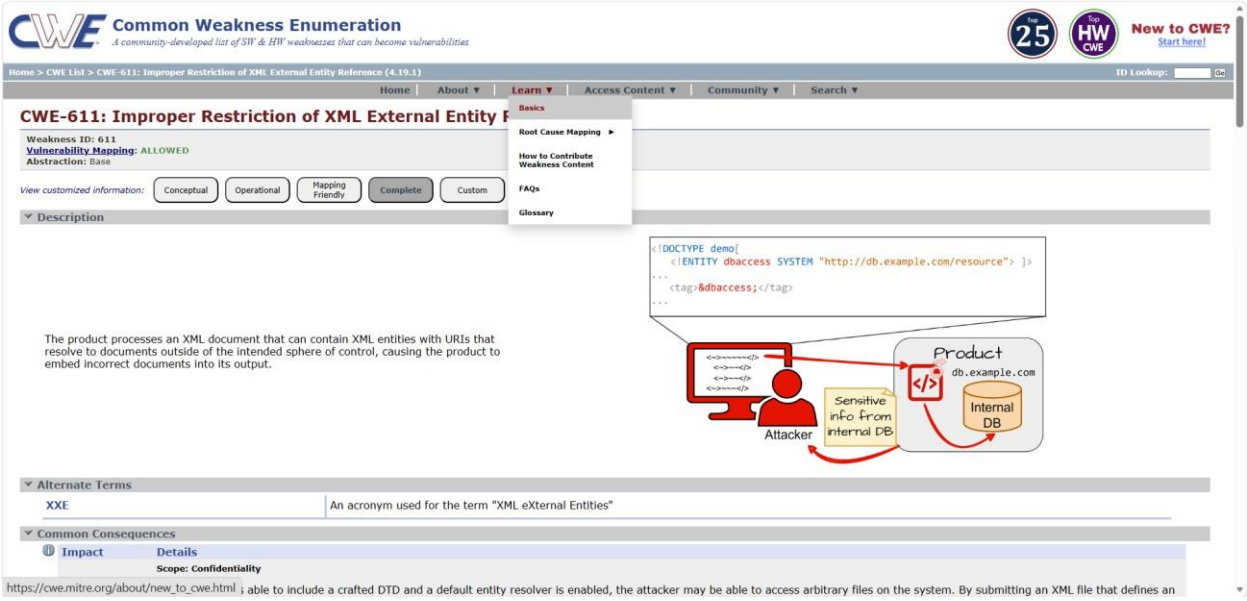
Vulnerability Path:

http://testfire.net/xmlparser.jsp

Steps to Reproduce:

- Access the XML processing endpoint in the application.
- Submit an XML request containing an external entity reference.
- The malicious XML includes a reference to a sensitive file on the server.

- If the application processes the XML without proper validation, the server will return the contents of the referenced file.



Recommendation:

- Disable external entity processing in XML parsers.
- Disable DTD processing in the XML configuration.
- Use secure XML parsing libraries that prevent XXE attacks.
- Validate and sanitize all XML input received by the application.
- Implement proper security configurations and access control mechanisms.

3. Vulnerability Name: Reliance on Insufficiently Trustworthy Component

CWE: 1357

OWASP Category: OWASP Top 10 – Software Supply Chain Failures

Description:

This vulnerability occurs when an application relies on external components, libraries, or dependencies that are not verified, outdated, or come from untrusted sources. Many modern

applications use open-source or third-party components to speed up development, but if these components contain vulnerabilities or malicious code, the entire application becomes at risk.

Attackers often target insecure or tampered third-party libraries to introduce backdoors, steal data, or compromise systems. When developers include such components without performing security validation or version checks, it can result in serious security threats. Even trusted libraries can become dangerous if not updated regularly, as attackers exploit known vulnerabilities in older versions.

Business Impact:

If exploited, attackers could gain unauthorized access to systems, inject malicious code, or manipulate data. Compromised third-party components can serve as entry points for large-scale supply chain attacks, leading to data breaches, financial losses, and loss of customer trust.

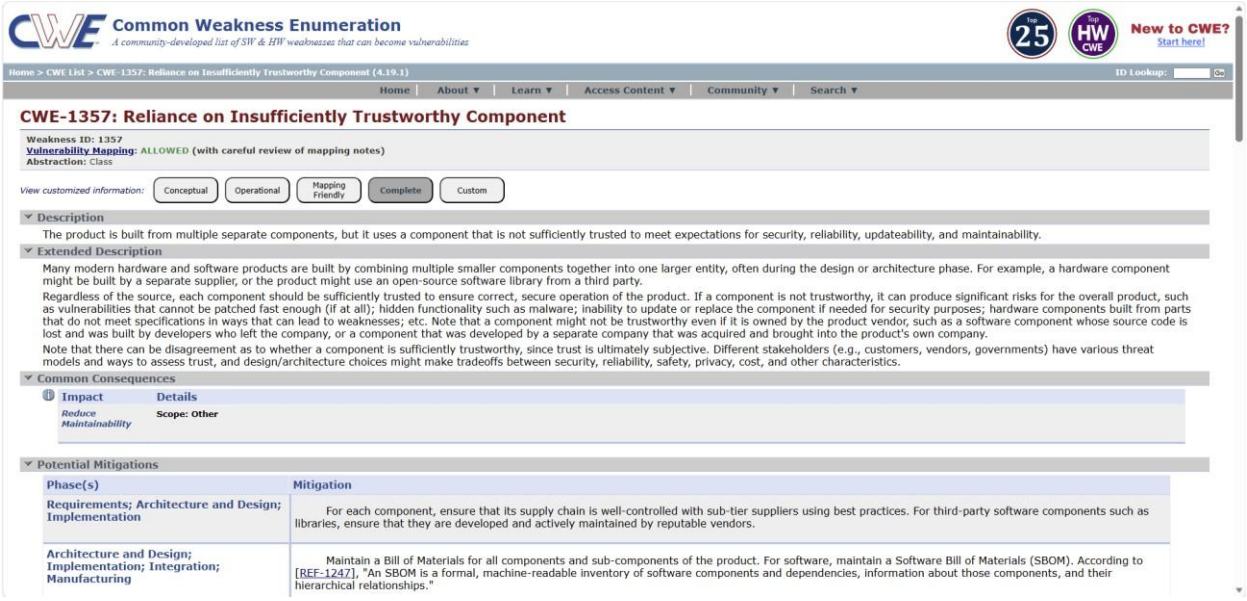
From a business perspective, reliance on unverified or outdated components may cause regulatory compliance violations, disrupt operations, and expose sensitive customer or company data. It can also damage the organization's reputation and increase maintenance and recovery costs.

Vulnerability Path:

http://testfire.net/index.jsp

Steps to Reproduce:

- Access the application's dependency or package configuration file (e.g., package. Json, pom.xml, or requirements.txt).
- Review the listed libraries and versions.
- Identify outdated or unverified components obtained from untrusted sources.
- Verify if any of these components contain known security vulnerabilities or lack digital signatures.



Common Weakness Enumeration
A community-developed list of SW & HW weaknesses that can become vulnerabilities

New to CWE? Start here!

Home > CWE List > CWE-1357: Reliance on Insufficiently Trustworthy Component (4.19.1)

ID Lookup:

Home | About ▾ | Learn ▾ | Access Content ▾ | Community ▾ | Search ▾

CWE-1357: Reliance on Insufficiently Trustworthy Component

Weakness ID: 1357
Vulnerability Mapping: ALLOWED **(with careful review of mapping notes)**
Abstraction: Class

View customized information: Conceptual | Operational | Mapping Friendly | Complete | Custom

Description

The product is built from multiple separate components, but it uses a component that is not sufficiently trusted to meet expectations for security, reliability, updateability, and maintainability.

Extended Description

Many modern hardware and software products are built by combining multiple smaller components together into one larger entity, often during the design or architecture phase. For example, a hardware component might be built by a separate supplier, or the product might use an open-source software library from a third party.

Regardless of the source, each component should be sufficiently trusted to ensure correct, secure operation of the product. If a component is not trustworthy, it can produce significant risks for the overall product, such as vulnerabilities that cannot be patched fast enough (if at all); hidden functionality such as malware; inability to update or replace the component if needed for security purposes; hardware components built from parts that do not meet specifications in ways that can lead to weaknesses; etc. Note that a component might not be trustworthy even if it is owned by the product vendor, such as a software component whose source code is lost and was built by developers who left the company, or a component that was developed by a separate company that was acquired and brought into the product's own company.

Note that there can be disagreement as to whether a component is sufficiently trustworthy, since trust is ultimately subjective. Different stakeholders (e.g., customers, vendors, governments) have various threat models and ways to assess trust, and design/architecture choices might make tradeoffs between security, reliability, safety, privacy, cost, and other characteristics.

Common Consequences

Impact	Details
Reduce Maintainability	**Scope: Other**

Potential Mitigations

Phase(s)	Mitigation
Requirements; Architecture and Design; Implementation	For each component, ensure that its supply chain is well-controlled with sub-tier suppliers using best practices. For third-party software components such as libraries, ensure that they are developed and actively maintained by reputable vendors.
Architecture and Design; Implementation; Integration; Manufacturing	Maintain a Bill of Materials for all components and sub-components of the product. For software, maintain a Software Bill of Materials (SBOM). According to [REF-1247], "An SBOM is a formal, machine-readable inventory of software components and dependencies, information about those components, and their hierarchical relationships."

Recommendation:

- Use only trusted and verified sources for third-party components.
- Regularly update all dependencies to the latest secure versions.
- Implement Software Composition Analysis (SCA) tools to detect vulnerable or untrusted components.
- Use digital signatures and integrity verification to ensure components haven't been tampered with.

4. Vulnerability Name: Insufficient Entropy

CWE: 331

OWASP Category: OWASP Top 10 – Cryptographic Failures

Description:

This vulnerability occurs when an application uses weak or predictable random number generation for security-related functions. Entropy refers to the level of randomness used when generating cryptographic values such as session IDs, encryption keys, tokens, or passwords.

If the randomness source is weak or predictable, attackers may be able to guess or reproduce these values. This can happen when applications use insecure random number generators, fixed seeds, or insufficient sources of randomness. As a result, attackers may predict session tokens, authentication keys, or other security tokens used by the system.

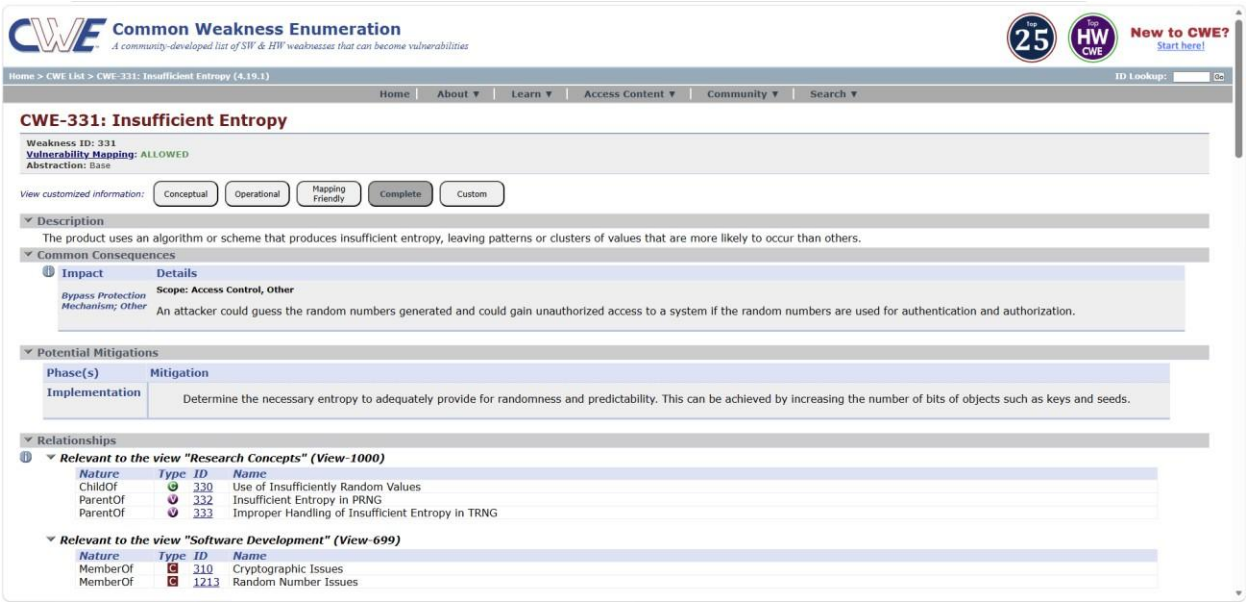
Business Impact:

If attackers are able to predict or guess cryptographic values, they may gain unauthorized access to user accounts or sensitive systems. Predictable session IDs or tokens can allow attackers to hijack active user sessions or impersonate legitimate users.

From a business perspective, this vulnerability may lead to account compromise, unauthorized data access, financial loss, and reputational damage. In severe cases, it can expose confidential information and weaken the overall security of the application.

Vulnerability Path:

http://testfire.net/login.jsp



Steps to Reproduce:

- Access the web application login page.
- Observe session IDs or tokens generated by the application.
- Refresh the page multiple times and analyse the generated values.
- If the values follow a predictable pattern or repeat frequently, the application may be using insufficient entropy.

Recommendation:

- Use secure cryptographic random number generators provided by trusted libraries.
- Avoid predictable values such as timestamps or sequential numbers for tokens.
- Generate session IDs and security tokens using strong entropy sources.
- Implement proper cryptographic standards for key and token generation.
- Regularly test and audit the application for cryptographic weaknesses.

5. Vulnerability Name: Public Static Field Not Marked Final

CWE: 500

OWASP Category: OWASP Top 10 – Injection

Description:

This vulnerability occurs when a public static variable in a program is not declared as final, allowing it to be modified by other classes or components within the application. Public static fields are accessible globally, and if they are not marked as final, their values can be changed during runtime.

When such variables store important configuration values, system settings, or security-related data, attackers or unauthorized components may manipulate them. This can lead to unexpected behavior, security bypass, or application instability. Improper use of public static variables can therefore introduce serious security risks.

Business Impact:

If attackers or malicious components modify public static variables, they may change the behavior of the application. For example, security checks could be bypassed, configuration values could be altered, or sensitive operations could be manipulated.

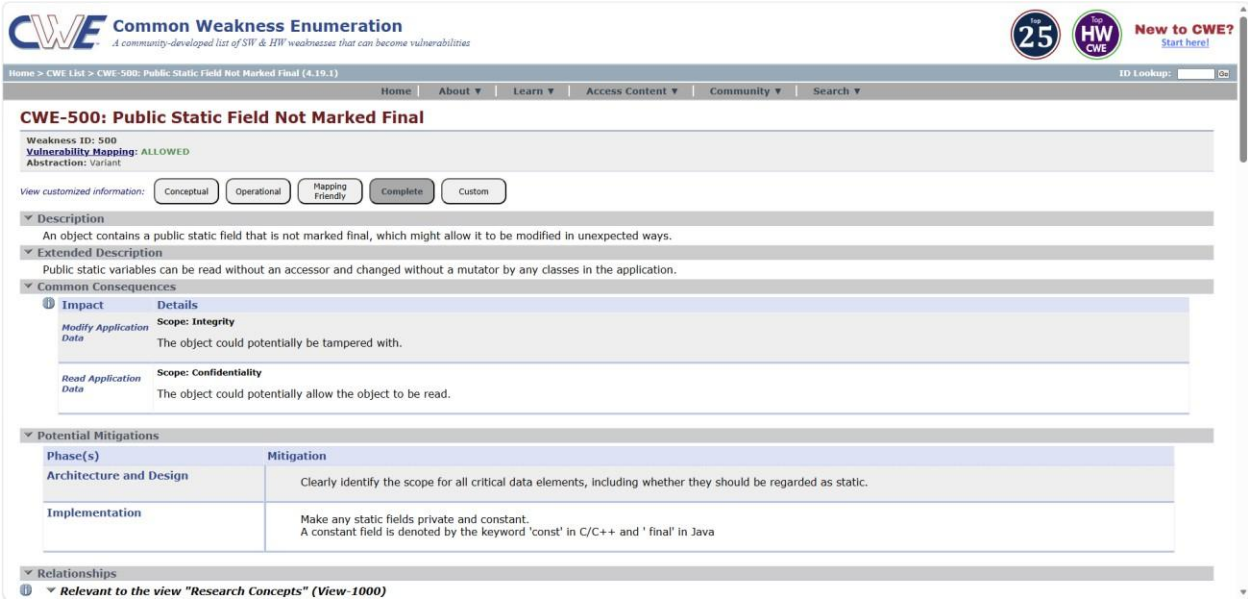
From a business perspective, this vulnerability may result in unauthorized access, system malfunction, data integrity issues, and potential security breaches. It can also reduce the reliability of the application and increase the risk of exploitation.

Vulnerability Path:

http://testfire.net/bank/transfer.jsp

Steps to Reproduce:

- Identify public static variables in the application source code.
- Check whether these variables are declared without the final keyword.
- Attempt to modify the variable value from another class or component.
- If the value can be changed during runtime, the vulnerability exists.



Recommendation:

- Avoid using public static variables for sensitive data or configuration values.
- Declare static variables as final if their value should not change.
- Use proper access control mechanisms such as private or protected fields.
- Implement secure coding practices and perform regular code reviews.

6. Vulnerability Name: Violation of Secure Design Principles

CWE: 657

OWASP Category: OWASP Top 10 – Insecure Design

Description:

This vulnerability occurs when an application is designed without following established security principles such as least privilege, defense in depth, or proper input validation. Secure design principles help ensure that applications are built with security considerations from the early stages of development.

When these principles are ignored, the system may contain weaknesses that attackers can exploit. For example, the application may allow excessive permissions, lack proper validation mechanisms, or fail to protect sensitive operations. Such design flaws can make the application more vulnerable to attacks and misuse.

Business Impact:

If secure design principles are not followed, attackers may exploit weaknesses in the system to gain unauthorized access, manipulate data, or bypass security controls. Poor design can expose sensitive information or allow unauthorized users to perform privileged operations.

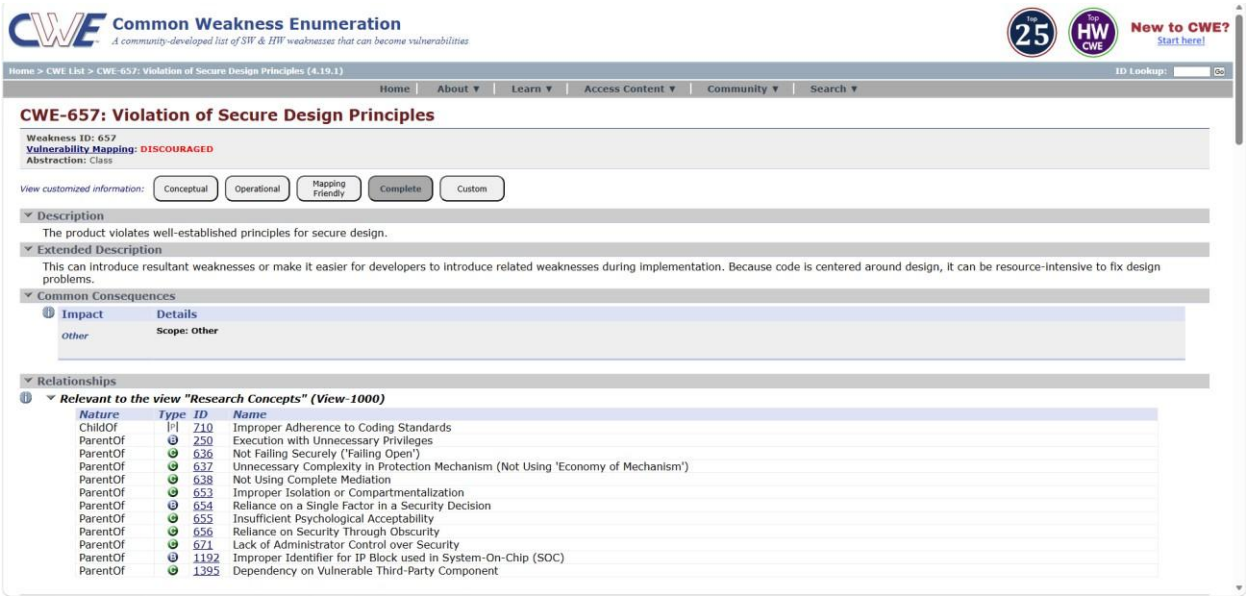
From a business perspective, this can lead to data breaches, financial loss, system compromise, and damage to the organization's reputation. It may also increase maintenance costs and require major redesign efforts to fix the security issues.

Vulnerability Path:

http://testfire.net/bank/account_summary.jsp

Steps to Reproduce:

- Access the application account summary page.
- Modify the account ID parameter in the URL.
- Attempt to access another user's account information.
- If the application displays the data without proper authorization checks, it indicates a violation of secure design principles.



Recommendation:

- Follow secure design principles such as least privilege and defense in depth.
- Implement proper authorization checks before granting access to sensitive data.
- Conduct secure architecture reviews during the design phase.
- Apply threat modeling to identify potential design weaknesses early in development.

7. Vulnerability Name: Use of Hard-coded Credentials

CWE: 798

OWASP Category: OWASP Top 10 – Authentication Failures

Description:

This vulnerability occurs when an application contains credentials such as usernames, passwords, or API keys that are directly embedded in the source code. Hard-coded credentials are often used by developers for testing or configuration purposes, but leaving them in the application can create serious security risks.

If attackers gain access to the source code or application files, they may discover these credentials and use them to access the system. Hard-coded credentials cannot be easily changed without modifying the application code, making them difficult to manage and secure. This increases the likelihood of unauthorized access.

Business Impact:

If attackers discover hard-coded credentials, they may gain direct access to application services, databases, or administrative functions. This can lead to unauthorized system access, data manipulation, or data theft.

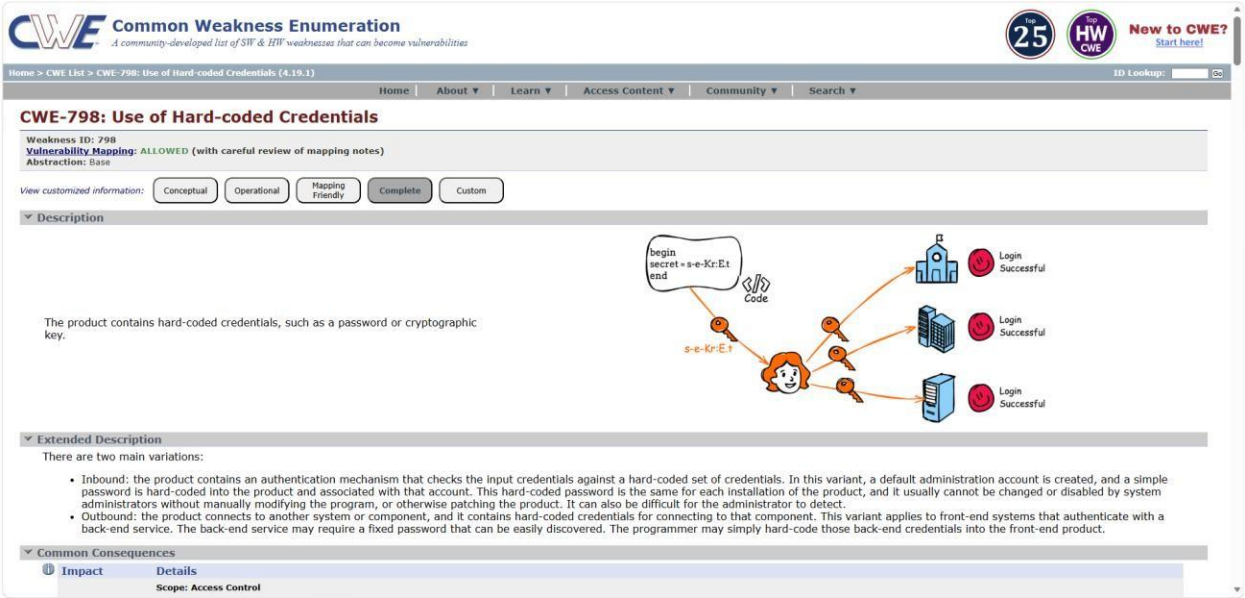
From a business perspective, this vulnerability can result in data breaches, financial loss, and damage to the organization's reputation. It may also allow attackers to bypass normal authentication controls and perform privileged operations within the system.

Vulnerability Path:
http://testfire.net/bank/profile.jsp

Steps to Reproduce:

- Review the application source code or configuration files.
- Look for credentials such as usernames, passwords, or API keys that are directly written in the code.

- If hard-coded credentials are found, they may allow unauthorized users to access the system.



Recommendation:

- Avoid storing credentials directly in source code.
- Use secure configuration management or environment variables to store sensitive information.
- Implement secure credential storage solutions such as secret management systems.
- Regularly review and audit application code to ensure sensitive credentials are not exposed.

8. Vulnerability Name: Replicating Malicious Code (Virus or Worm)

CWE: 509

OWASP Category: OWASP Top 10 – Software or Data Integrity Failures

Description:

This vulnerability occurs when malicious code such as a virus or worm is able to replicate itself within a system or application. Replicating malicious code can spread automatically across files,

applications, or network systems without the user's knowledge. These programs are designed to copy themselves and infect other systems or components.

Such malicious code may enter the system through insecure software updates, compromised files, or untrusted components. Once inside the system, it can replicate and spread rapidly, affecting multiple parts of the application or network environment.

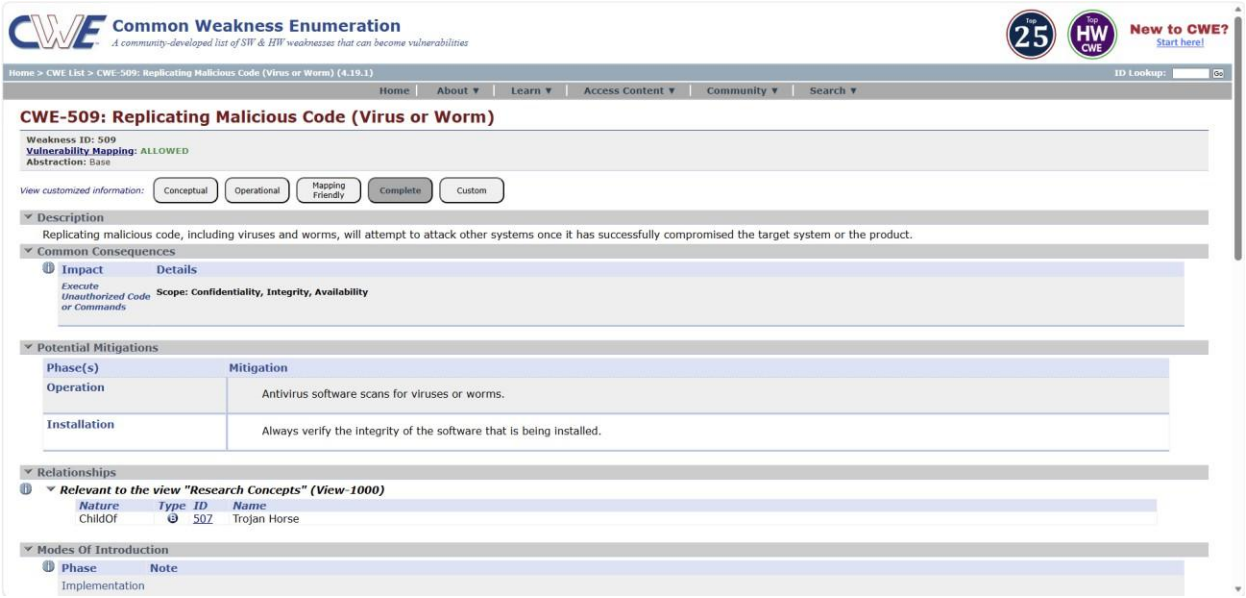
Business Impact:

If malicious code replicates within a system, it can damage files, steal sensitive information, or disrupt normal system operations. In some cases, the malware may spread across the entire network, affecting multiple users and systems.

From a business perspective, this can lead to system downtime, data loss, security breaches, and significant financial damage. It may also harm the organization's reputation and require costly recovery efforts to remove the malware and restore affected systems.

Vulnerability Path:

http://testfire.net/bank/transaction_history.jsp



Steps to Reproduce:

- Access the transaction history page.
- Attempt to upload or execute a file from an untrusted or external source.
- If the application allows execution of the file without proper validation, malicious code may spread within the system.

Recommendation:

- Verify the integrity of all software components and updates before execution.
- Implement code signing and integrity checks to ensure that only trusted code runs in the system.
- Use anti-malware protection and monitoring tools to detect suspicious activity.
- Restrict file execution from untrusted sources and apply proper input validation.

9. Vulnerability Name: Information Loss of Omission

CWE: 221

OWASP Category: OWASP Top 10 – Security Logging and Alerting Failures

Description:

This vulnerability occurs when an application fails to properly log important security events or system activities. Information loss of omission means that certain critical actions, errors, or security-related events are not recorded in the system logs.

Without proper logging, it becomes difficult to detect suspicious behavior, security incidents, or unauthorized access attempts. Attackers may exploit this weakness to perform malicious activities without leaving evidence in the system logs.

Business Impact:

If important security events are not logged, organizations may fail to detect attacks, unauthorized access, or misuse of system resources. This lack of visibility makes incident investigation and forensic analysis very difficult.

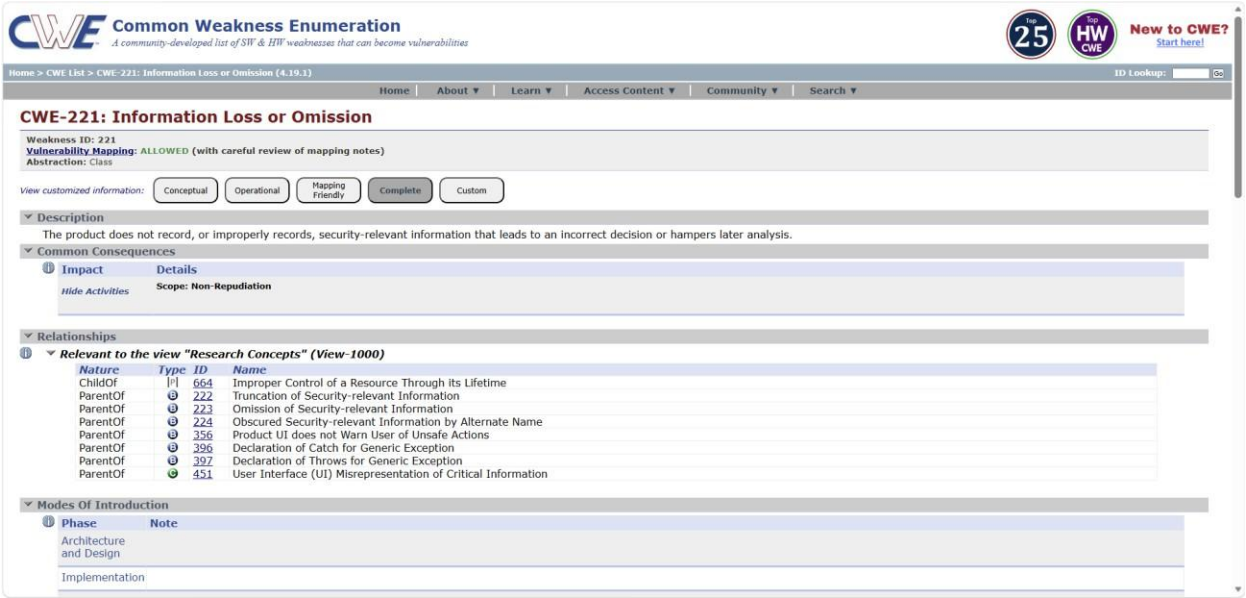
From a business perspective, this can lead to delayed detection of cyber attacks, increased security risks, data breaches, and compliance issues. It may also impact the organization's ability to respond quickly to security incidents.

Vulnerability Path:

http://testfire.net/bank/login.jsp

Steps to Reproduce:

- Access the login page of the application.
- Enter incorrect login credentials multiple times.
- Check whether the application records these failed login attempts in the system logs.
- If the system does not log these events, it indicates an information loss of omission vulnerability.



Recommendation:

- Implement proper security logging mechanisms to record all important system events.
- Ensure that login attempts, authentication failures, and sensitive actions are logged.
- Use real-time monitoring and alerting systems to detect suspicious activity.
- Regularly review and audit logs to identify potential security incidents.

10. Vulnerability Name: Omitted Break Statement in Switch

CWE: 484

OWASP Category: OWASP Top 10 – Mishandling of Exceptional Conditions

Description:

This vulnerability occurs when a switch statement in the program does not include a break statement after a case block. As a result, the program continues executing the next case statements unintentionally, which is known as "fall-through" behavior.

If developers forget to include a break statement where required, it may lead to incorrect logic execution, unexpected results, or unintended actions within the application. This can cause the system to process operations that were not originally intended for a particular case condition.

Business Impact:

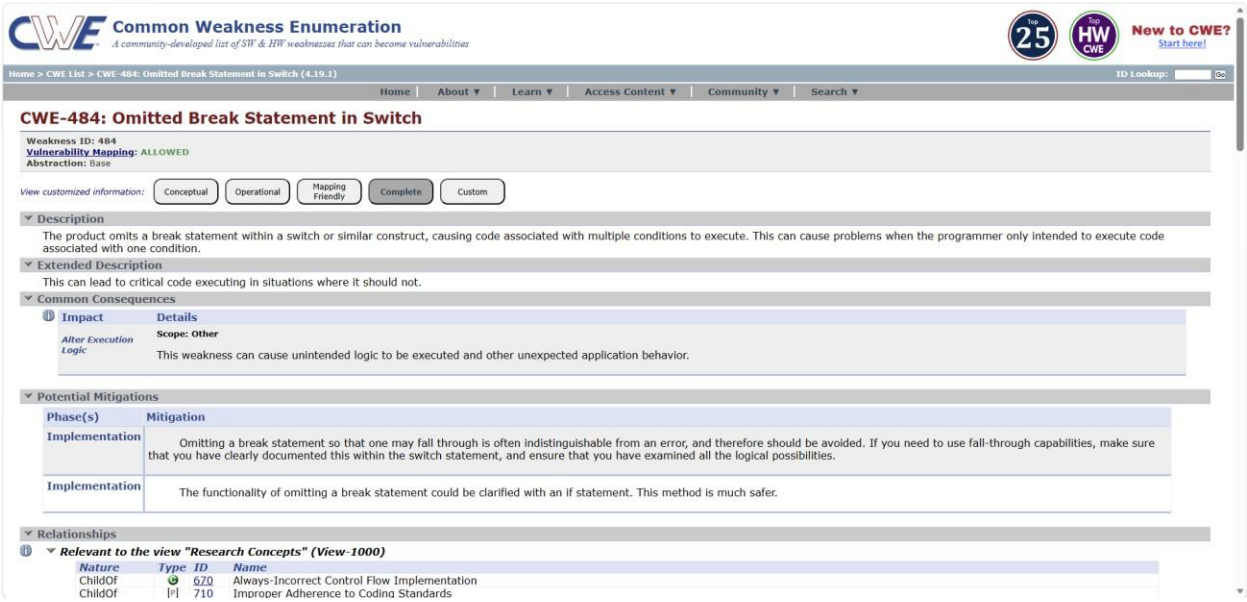
An omitted break statement can cause the application to execute multiple case conditions, leading to incorrect processing of data or system actions. In some cases, this may allow users to bypass certain restrictions or trigger unintended operations.

From a business perspective, this may result in system errors, incorrect data processing, or unauthorized functionality being executed. Over time, such flaws can affect application reliability and security.

Vulnerability Path:
http://testfire.net/bank/billpay.jsp

Steps to Reproduce:

- Access the bill payment page of the application.
- Perform different actions that trigger the switch case logic.
- Observe whether the system executes additional operations beyond the intended action.
- If multiple case conditions are executed due to missing break statements, the vulnerability exists.



Common Weakness Enumeration
A community-developed list of SW & HW weaknesses that can become vulnerabilities

New to CWE? Start here!

Home > CWE List > CWE-484: Omitted Break Statement in Switch (4.19.1)

ID Lookup:

Home | About ▼ | Learn ▼ | Access Content ▼ | Community ▼ | Search ▼

CWE-484: Omitted Break Statement in Switch

Weakness ID: 484
Vulnerability Mapping: ALLOWED
Abstraction: Base

View customized information: Conceptual | Operational | Mapping Friendly | Complete | Custom

Description

The product omits a break statement within a switch or similar construct, causing code associated with multiple conditions to execute. This can cause problems when the programmer only intended to execute code associated with one condition.

Extended Description

This can lead to critical code executing in situations where it should not.

Common Consequences

Impact	Details
Alter Execution Logic	Scope: Other This weakness can cause unintended logic to be executed and other unexpected application behavior.

Potential Mitigations

Phase(s)	Mitigation
Implementation	Omitting a break statement so that one may fall through is often indistinguishable from an error, and therefore should be avoided. If you need to use fall-through capabilities, make sure that you have clearly documented this within the switch statement, and ensure that you have examined all the logical possibilities.
Implementation	The functionality of omitting a break statement could be clarified with an if statement. This method is much safer.

Relationships

Relevant to the view "Research Concepts" (View-1000)

Nature	Type	ID	Name
ChildOf	C	670	Always-Incorrect Control Flow Implementation
ChildOf	P	710	Improper Adherence to Coding Standards

Recommendation:

- Always include break statements in switch-case blocks unless fall-through behavior is explicitly required.
- Perform code reviews and static code analysis to detect missing break statements.
- Implement secure coding practices and proper testing during development.

Stage 2

Overview :

Context-aware access control is a modern security approach designed to manage and regulate user access to systems, networks, and applications based on multiple contextual factors. Unlike traditional access control methods that depend only on usernames and passwords, this approach evaluates additional parameters such as user identity, device posture, location, and application risk level before granting access. This helps organizations ensure that only authorized users can access sensitive resources under secure conditions.

The system works by continuously analyzing different contextual attributes when a user attempts to access an application or network resource. First, the system verifies the user identity through authentication mechanisms. After successful authentication, the system checks the device posture, which includes security conditions such as operating system updates, antivirus status, and device compliance with security policies.

Another important factor considered in context-aware access control is the user's location. The system evaluates whether the access request is coming from a trusted network or an unusual geographical location. If the location appears suspicious or outside the allowed region, additional security verification may be required before granting access.

The system also evaluates the risk level of the requested application. Applications that contain sensitive or confidential data are assigned a higher risk level and require stronger authentication and stricter access policies. Based on all these contextual factors, the system dynamically decides whether to allow access, restrict certain actions, or completely deny the request.

After analyzing the contextual information, the system generates security logs and access reports that help administrators monitor user activities and detect potential security threats. This approach improves overall cybersecurity by reducing unauthorized access and protecting critical organizational resources.

Overall, context-aware access control plays an important role in modern cybersecurity environments by providing intelligent and adaptive security mechanisms. By combining identity verification, device security checks, location analysis, and application risk evaluation, the system ensures a more secure and reliable access control framework.

Target Website: testfire.net
Target IP Address: 44.228.249.3

List of vulnerabilities:

S. NO	VULNERABILITY NAME	SEVERITY	PLUGINS
1.	Cross-site scripting (stored)	Low	Web application Scanner
2.	Broken Access Control	Medium	SSH weak HMAC Algorithms Supported
3.	SQL Injection	Medium	N/A
4.	Broken Authentication	Low	HSTS Missing from Server
5.	Insecure Direct Object Reference	Info	Service Detection
6.	Security Misconfiguration	Low	SSL cipher Block Chaining cipher suites
7.	Cross-Site Request Forgery	Low	HTTP Form- Based Authentication Detected
8.	Cleartext Transmission of Sensitive Information	Medium	Cleartext Transmission of Sensitive Information over HTTP
9.	ClickJacking	Info	HTML Rendering Layer Vulnerability

10.	Webserver allows password auto-completion	Low	SSL/TLS Diffie-Hellman Modulus<=1024 bits(Logjam)

Report:

1. Vulnerability Name: Cross site scripting (stored)
Severity: Low
Plugin: web application scanner
Port: N/A

Description: It occurs when a malicious script is injected directly into a vulnerable web application. Reflected XSS involves the reflection of a malicious script of a web application, onto a user's browser.

Solution:

- When interacting with databases, use parameterized statements or prepared statements to avoid SQL injection, which can be a vector for XSS.
- Sanitize and validate all user inputs and ensure that any data displayed on the web page properly encoded. Use output encoding libraries or functions to prevent script injection.

Business Impact: The application stores dangerous data in a database, message forum, visitor log, or other trusted data store. Later, the dangerous data is subsequently read back into the application and included in dynamic content. From an attacker's perspective, the optimal place to inject malicious content is in an area that is displayed to either many users or particularly interesting users. Interesting users typically have elevated privileges in the application or interact with sensitive data that is valuable to the attacker. If one of these users executes malicious content, the attacker may be able to perform privileged operations on behalf of the user or gain access to sensitive data belonging to the user.

2. **Vulnerability Name:** Broken access control

Severity: medium

Plugin: SSH weak HMAC Algorithms supported

Port: N/A

Description: When access control checks are not applied consistently - or not at all - users are able to access data or perform actions that they should not be allowed to perform. This can lead to a wide range of problems, including information exposures, denial of service, and arbitrary code execution.

Solution:

- Ensure that the application has a robust authentication and authorization mechanism in place. Users should be authenticated before any sensitive actions are performed, and authorization checks should be conducted to verify that users have the necessary permissions for the requested operation.
- Ensure that session management is secure. Use strong session IDs, enforce session timeouts, and regenerate session tokens after login. Implement session fixation protection to prevent session hijacking.

Business Impact: wide-ranging business impact, including financial losses, reputation damage, legal consequences, operational disruption, and loss of customer trust. Addressing this vulnerability is crucial to protect the organization and its stakeholders from these potential negative effects.

3. **Vulnerability Name:** SQL injection

Severity: medium

Plugin: N/A (typically would be a security testing tool)

Port: port 80/443

Description: The product constructs all or part of an SQL command using externally influenced input from an upstream component, but it does not neutralize or incorrectly neutralizes special elements that could modify the intended SQL command when it is sent to a downstream component.

Solution:

- If parameterized statements are not feasible, use proper input validation and escaping mechanisms to sanitize user inputs before they are used in SQL queries. This helps prevent malicious code injection.
- Utilize stored procedures for database operations. This can help separate SQL code from application code and reduce the risk of SQL injection.
- Implement strict input validation for all user inputs, ensuring that data adheres to the expected format and structure.

Business Impact: In summary, it is crucial to underscore that CWE-89, known as SQL Injection, can exert a profound and diverse business impact. This encompasses critical facets such as data breaches, financial setbacks, harm to reputation, legal ramifications, and operational turmoil. Hence, the imperative of preventing and remedying SQL injection vulnerabilities cannot be overstated, as it is indispensable for fortifying the security and continuity of an organization's applications and data.

4. **Vulnerability Name**: Broken authentication

Severity: Low

Plugin: HSTS Missing from HTTPS Server (plugin id:142960)

Port: port 443

Description: The system's authorization functionality does not prevent one user from gaining access to another user's data or record by modifying the key value identifying the data.

Solution:

- Implement strong authentication mechanisms, such as multi-factor authentication (MFA) or two-factor authentication (2FA), to ensure that only authorized users can access the system.
- Implement proper authorization controls to ensure that users have the necessary permissions to access specific functions or data. Use role-based access control (RBAC) to manage user privileges effectively.

Business Impact: To effectively mitigate the business impact of CWE-285, it is imperative that organizations place a strong emphasis on bolstering their authentication and session

management practices. This should encompass the adoption of multi-factor authentication, the secure storage of credentials, meticulous session handling, and a commitment to conducting routine security assessments and testing. The rectification of these vulnerabilities stands as a critical imperative, safeguarding sensitive data, user identities, and the organization's overarching security stature and reputation.

5. **Vulnerability Name:** Insecure Direct object Reference
Severity: Info
Plugin: Service Detection (plugin id:22964)
Port: port 80/443

Description: Insecure Direct Object Reference (IDOR) is a vulnerability that arises when attackers can access or modify objects by manipulating identifiers used in a web application's URLs or parameters. It occurs due to missing access control checks, which fail to verify whether a user should be allowed to access specific data.

Solution:

- Utilize parameterized queries or prepared statements when interacting with databases. This prevents user-controlled input from being executed as SQL code.
- Implement a Content Security Policy to mitigate the risk of cross-site scripting (XSS) attacks, which can be used to execute unauthorized queries.
- Implement account lockout mechanisms that temporarily suspend accounts after a specified number of unauthorized access attempts to prevent brute force attacks.

Business Impact: To effectively mitigate the business impact of CWE-639, organizations must make it a top priority to fortify their access control and authorization mechanisms. This entails the implementation of robust security measures, the regular conduct of comprehensive security assessments, and the deployment of intrusion detection systems to promptly identify and counter unauthorized access attempts. These measures stand as critical in the defence of sensitive data, the overall security of systems, and the preservation of the organization's esteemed reputation.

6. **Vulnerability Name:** Security Misconfiguration

Severity: low

Plugin: SSL cipher Block chaining cipher suites

Port: port 443

Description: The product specifies permissions for a security-critical resource in a way that allows that resource to be read or modified by unintended actors.

Solution:

- Implement a change management process to document and review all changes to system configurations. This helps prevent unauthorized or unintended alterations.
- Follow security guidelines provided by software and hardware vendors to ensure secure configurations.
- Regularly perform compliance checks against industry standards and regulations to ensure that configurations remain in compliance with security requirements.

Business Impact: To effectively lessen the business consequences stemming from CWE-732, organizations must place a strong emphasis on bolstering their configuration management practices. This involves the establishment of secure default settings, the routine conduct of comprehensive security evaluations, and the deployment of intrusion detection systems to promptly detect and address misconfigurations and vulnerabilities. These actions are of paramount importance in safeguarding sensitive data, fortifying system security, and preserving the organization's esteemed reputation.

7. **Vulnerability Name:** Cross-Site Request Forgery

Severity: Low

Plugin: HTTP Form-Based Authentication Detected

Port: port 8080/443

Description: Cross-Site Request Forgery (CSRF) is an attack that forces an end user to execute unwanted actions on a web application in which they're currently authenticated. With a little help of social engineering (such as sending a link via email or chat), an attacker may trick the

users of a web application into executing actions of the attacker's choosing. If the victim is a normal user, a successful CSRF attack can force the user to perform state changing requests like transferring funds, changing their email address, and so forth. If the victim is an administrative account, CSRF can compromise the entire web application.

Solution:

- Implement Synchronizer Token Patterns to include unique, randomly generated tokens in each HTTP request and validate these tokens on the server side to verify the authenticity of the request.
- Utilize the Same Site attribute in cookies to restrict their usage to same-site requests only, preventing them from being sent along with cross-site requests and effectively mitigating CSRF attacks.

Business Impact: This vulnerability may allow attackers to perform unauthorized actions on behalf of authenticated users without their knowledge. An attacker could trick users into submitting malicious requests that modify account settings or application data. If exploited through an administrative account, it could lead to unauthorized changes in the system. This may impact data integrity, system functionality, and user trust in the application.

8 .Vulnerability Name: Cleartext Transmission Sensitive Information
Severity: Medium
Plugin: Cleartext Transmission of Sensitive Information over HTTP
Port: port 443

Description: The product transmits sensitive or security-critical data in cleartext in a communication channel that can be sniffed by unauthorized actors. Many communication channels can be "sniffed" (monitored) by adversaries during data transmission. For example, in networking, packets can traverse many intermediary nodes from the source to the destination, whether across the internet, an internal network, the cloud, etc. Some actors might have privileged access to a network interface or any link along the channel, such as a router, but they might not be authorized to collect the underlying data. As a result, network traffic could be sniffed by adversaries, spilling security-critical data.

Solution:

- Educate users and employees on secure data handling practices, such as recognizing secure websites (look for "https") and avoiding insecure Wi-Fi networks.
- Employ data masking or redaction techniques to replace sensitive data with placeholders or cryptographic representations during transmission.
- Implement encryption for data in transit. Use secure encryption protocols such as TLS/SSL to protect sensitive information during transmission.
- Use secure communication protocols for transmitting data, such as HTTPS for web applications, and ensure that the selected protocols are kept up to date.

Business Impact: To effectively reduce the business consequences associated with CWE-319, organizations must prioritize the adoption of secure data transmission practices. This includes the utilization of encryption and robust, secure protocols. The routine conduct of security assessments and testing is pivotal in pinpointing and remedying vulnerabilities linked to data transmission. Furthermore, the education of users on secure data handling practices plays a vital role in proactively preventing data exposure incidents. These actions are of paramount importance in the protection of sensitive data and in preserving the trust of both customers and partners.

9. **Vulnerability Name:** Clickjacking

Severity: Info
Plugin: HTML Rendering Layer Vulnerability
Port: port 80/443

Description: The web application does not restrict or incorrectly restricts frame objects or UI layers that belong to another application or domain, which can lead to user confusion about which interface the user is interacting with.

Solution:

- Set the X-Frame-Options HTTP response header to deny or same-origin to control how your site can be framed. This is supported by most modern browsers.

- Utilize frame-busting JavaScript code in web applications to prevent the embedding of your site within malicious frame's. This code can disrupt clickjacking attempts.
- Implement a Content Security Policy to restrict which domains can embed your site in iframes. This can help prevent unauthorized framing.
- Implement additional security controls to prevent UI redress attacks, such as clickjacking, within your web application.

Business Impact: To effectively reduce the business consequences resulting from CWE-1021, organizations must prioritize the implementation of protective measures, such as frame-busting code. Simultaneously, educating users on safe browsing practices plays a crucial role in preventing clickjacking incidents. Additionally, routine security assessments and testing are pivotal for identifying and mitigating vulnerabilities associated with clickjacking. These actions are of paramount importance in upholding user trust, ensuring data protection, and safeguarding the organization's reputation

10. Vulnerability Name: web server allows password auto-completion
Severity: Low
Plugin: SSL/TLS Diffie-Hellman Modulus <= 1024 Bits (Logjam)
Port: port 443

Description: Weaknesses in this category are related to the design and implementation of data confidentiality and integrity. Frequently these deal with the use of encoding techniques, encryption libraries, and hashing algorithms. The weaknesses in this category could lead to a degradation of the quality data if they are not addressed.

Solution:

- Implement secure password storage mechanisms, such as using strong and salted cryptographic hashing algorithms like crypt or script.
- Implement robust encryption key management practices, including secure key storage, key rotation, and access controls.
- Utilize well-established and proven cryptographic libraries and algorithms for encryption and decryption.

- Use secure encryption protocols, like TLS/SSL, for data transmission over networks to protect data in transit

Business Impact: To effectively lessen the business consequences associated with CWE-310,organizations must prioritize the adoption of secure cryptographic practices. This entails ensuring proper password storage and robust encryption key management. Simultaneously, conducting routine security assessments and testing is pivotal in detecting and mitigating vulnerabilities related to cryptographic issues. Striving for compliance with pertinent data protection regulations and industry standards is equally essential. These actions are of paramount importance in the protection of sensitive data, the upholding of user trust, and the preservation of the organization's esteemed reputation.

Stage 3

Report

Tittle : DESIGN AND APPLY CONTEXT-AWARE ACCESS CONTROLS BASED ON USER IDENTITY, DEVICE POSTURE, LOCATION, AND APPLICATION RISK

Security Operations Center (SOC)

In a Security Operations Center (SOC), context-aware access control is used to protect applications and network resources. When a user tries to access an application, the system first verifies the user's identity using authentication services like Microsoft Azure Active Directory. The SOC also checks the security condition of the user's device to ensure it follows the organization's security policies. In addition, the user's location and network are analyzed to detect any unusual or suspicious login attempts. Applications that contain sensitive data are given stronger security controls. Security platforms such as Zscaler help SOC teams monitor access and enforce these policies. This approach helps prevent unauthorized access and improves overall network security.

SOC – cycle

The Security Operations Center (SOC) cycle is a continuous process used to manage and respond to cybersecurity threats in an organization. It starts with monitoring, where security tools observe network activities and collect system logs. Next is detection, where suspicious activities or possible attacks are identified. After that, the SOC team performs investigation to understand the threat and its impact. Then comes the response stage, where actions are taken to stop and remove the threat. Finally, the recovery stage restores affected systems and brings operations back to normal. Security tools such as Zscaler and IBM QRadar help SOC teams monitor and manage this entire cycle effectively.

Security Information and Event Management (SIEM)

Security Information and Event Management (SIEM) is a cybersecurity system that collects and analyzes security logs from different devices and applications in a network. It helps security teams monitor activities, detect suspicious behavior, and respond to cyber threats quickly.

SIEM gathers log data from servers, network devices, applications, and authentication systems, then analyzes this information to find unusual activities or possible attacks. When a threat is detected, the system generates alerts so the security team can investigate and take action. Tools such as IBM QRadar are commonly used SIEM platforms that help organizations improve security monitoring and threat detection.

SIEM Cycle

The SIEM cycle is a process used to monitor and manage security events in an organization's network. It begins with log collection, where security data is gathered from servers, applications, and network devices. Next, the data is analyzed to identify unusual or suspicious activities. After analysis, the system correlates events from different sources to detect possible security threats. If any threat or abnormal behavior is found, the SIEM system generates alerts so the security team can investigate and respond quickly. Tools like IBM QRadar help organizations manage this cycle and improve security monitoring.

Malware Information Sharing Platform (MISP)

Malware Information Sharing Platform (MISP) is an open-source platform used to share information about cyber threats and malware. It helps organizations collect, store, and share threat intelligence such as malware details, vulnerabilities, and attack indicators. Security teams use MISP to learn about new cyber threats and protect their systems more effectively. By sharing threat information with other organizations, MISP helps improve early detection and prevention of cyber attacks. Tools like MISP Threat Intelligence Platform help security teams strengthen their cybersecurity and respond to threats faster.

Your college network information

The college network connects computers, servers, routers, and switches so that students and staff can access the internet, learning resources, and college systems.

This network is used for activities such as accessing online study materials, submitting assignments, and managing administrative data. To keep the network secure, proper security measures like user authentication, monitoring, and access control are important. These security practices help protect sensitive information and ensure that only authorized users can access the college network.

How you think you deploy soc in your college

A Security Operations Center (SOC) can be deployed in the college to monitor and protect the network from cyber threats. Security tools can collect logs from computers, servers, and network devices to monitor activities.

These logs can be analyzed to detect suspicious behavior or attacks. Authentication systems can also be monitored to track user access to college applications. Security platforms such as IBM QRadar can help analyze logs and generate alerts for threats. By using SOC, the college can improve network security, detect cyber threats early, and respond quickly to security incidents.

Threat intelligence

Threat intelligence is the process of collecting and analyzing information about cyber threats and attacks. It helps organizations understand how cybercriminals operate and what types of attacks may happen. Security teams use this information to identify possible risks, detect threats early, and improve their security systems. Platforms such as MISP Threat Intelligence Platform help organizations share and analyze threat information, making it easier to prevent cyber attacks and protect networks and data.

Incident response

Incident response is the process of handling and managing cybersecurity incidents or attacks. It helps organizations quickly identify and investigate security threats.The security team then takes action to stop the attack and remove the threat.Finally, the affected systems are restored and steps are taken to prevent future incidents.

Qradar & understanding about tool

IBM QRadar is a Security Information and Event Management (SIEM) tool used to monitor and analyze security events in a network. It collects log data from different devices, servers, and applications. The tool analyzes this data to detect suspicious activities and possible cyber threats. QRadar also generates alerts and provides dashboards to help security teams quickly investigate and respond to security incidents.

Conclusion

Stage 1

Stage 1 focused on understanding how Zscaler can be integrated with identity providers like Microsoft Azure Active Directory. This stage helped in learning about secure authentication, Zero Trust access, and identifying security risks in cloud environments.

Stage 2

Stage 2 focused on vulnerability scanning and security analysis using tools like Nessus Vulnerability Scanner. It helped identify security weaknesses in websites and provided recommendations to fix those vulnerabilities.

Stage 3

Stage 3 explained advanced cybersecurity concepts such as Security Operations Center (SOC), SIEM monitoring, threat intelligence, and incident response. These technologies help organizations monitor networks, detect cyber threats, and respond quickly to security incidents to improve overall cybersecurity.

Future Scope :

Stage 1

In the future, organizations can improve secure access by integrating more advanced Zero Trust technologies with platforms like Zscaler and identity services such as Microsoft Azure Active Directory. This will help provide stronger authentication, better user access control, and improved protection for cloud applications.

Stage 2

Future improvements can include advanced vulnerability scanning and automated security testing using tools like Nessus Vulnerability Scanner. This will help organizations identify security weaknesses faster and fix them before attackers exploit them.

Stage 3

Future improvements can include advanced vulnerability scanning and automated security testing using tools like Nessus Vulnerability Scanner. This will help organizations identify security weaknesses faster and fix them before attackers exploit them.

Topics explored :

- Zero Trust Architecture
- Identity and Access Management
- Context-Aware Access Control
- Cloud Security Concepts
- Vulnerability Assessment
- Web Application Security
- Security Risk Identification
- Vulnerability Analysis

Tools explored :

- Zscaler
- Microsoft Azure Active Directory
- Nessus Vulnerability Scanner
- IBM QRadar
- MISP Threat Intelligence Platform

THE END